HiLSVA: Design and Evaluation of a Human-in-the-Loop Agentic System for Scientific Visualization

Kuangshi Ai, Patrick Phuoc Do, and Chaoli Wang

Abstract—Large language model (LLM) agents enable natural language interaction for scientific visualization (SciVis). Still, prior systems have essentially prioritized autonomy over human analytical control, thereby limiting transparency and human oversight. We present HiLSVA, a human-in-the-loop agentic system that supports mixed-initiative SciVis workflows. HiLSVA integrates a plan-first multi-agent architecture with explicit human oversight, stepwise provenance tracking, and learn-at-test-time adaptation from user feedback. The system supports fluid handoff between humans and agents through both natural language and direct manipulation of visualizations, while sandboxed execution ensures safe, reproducible workflows. In doing so, HiLSVA reframes agentic SciVis as a collaborative process that augments, rather than replaces, human analytical reasoning. We evaluate HiLSVA through representative case studies and a controlled user study with twelve participants of varying expertise across multiple autonomy settings. Results show that mixed-initiative interaction improves task completion, user control, and workflow transparency across different levels of user expertise, while revealing a tradeoff between execution efficiency and human oversight. These findings highlight the importance of human-centered design in agentic SciVis and guide the development of future collaborative visualization systems. We encourage readers to explore our demo video, case studies, and source code at https://hilsva.github.io/.

Index Terms—Scientific visualization, LLM agents, human–agent collaboration, human-centered computing

arXiv:2606.26614v2 [cs.HC] 16 Jul 2026

1 Introduction

Advances in large language models (LLMs) have made complex human–computer interactions more intuitive and accessible. They have reshaped general computer use by enabling conversational agents to assist with everyday tasks such as file management, email drafting, and troubleshooting [6, 30, 76, 82]. In web navigation, LLM agents can search, filter, and summarize online content, assisting users in making decisions or even completing tasks such as booking tickets on their behalf [37, 41]. Beyond these broad applications, LLMs are transforming domains that once demanded specialized expertise. In software engineering, they can generate code, explain concepts, and aid debugging, lowering barriers for both novices and experts [34, 42]. In visualization, they bridge the gap between analytical intent and implementation, allowing users to state goals in natural language and obtain executable workflows [44, 85]. Building on this trajectory, autonomous LLM-driven agents now enable natural-language instructions to be translated directly into sophisticated visualizations.

In scientific visualization (SciVis), several recent systems exemplify this paradigm as we witness the shift from generative AI [72] to agentic AI. ChatVis [53] introduces an iterative LLM agent that generates and refines ParaView Python scripts through retrieval-augmented prompting and error-driven feedback, thereby improving the robustness of code generation. ParaView-MCP [43] adopts the *model context protocol* (MCP) to directly connect multimodal LLMs with ParaView's Python API, enabling seamless collaboration among the user, agent, and visualization environment. Beyond direct tool control, NLI4VolVis [5] advances natural language interaction for volumetric data by combining LLM multi-agents, editable 3D Gaussian splatting [69], and semantic segmentation, supporting open-vocabulary querying and real-time scene editing. VizGenie [11] introduces a self-improving framework that dynamically generates, validates, and integrates new visualization modules, enabling adaptive, feature-based exploration. These systems illustrate an emerging class of **LLM-based SciVis agents**, which can be broadly defined as *AI agents that interpret user intent expressed in natural language and autonomously interact with visualization pipelines to produce outputs aligned with user-specified analysis goals* [2, 15].

However, existing works tend to emphasize autonomy and task completion, often sidelining the central role of human analysts. As Dhanoa et al. [15] argued, visualization is fundamentally human-centered, emphasizing reasoning, interpretability, and accountability rather than automation alone. The critical challenge is not simply to build more autonomous SciVis agents, but to balance human analytical control, encompassing how analysts delegate, coordinate, and verify agent actions, with machine autonomy. This balance requires allowing humans to decide when to grant or withdraw initiative, maintaining transparency into agents' reasoning processes, and preserving authority over interpretation and decision-making. After all, the goal of SciVis is not automation for its own sake, but to empower people to conduct scientific analysis, derive meaningful insights, and make informed decisions.

Recent work in other domains highlights the promise of **human-in-the-loop agentic systems**. Senoner et al. [57] showed that explainable AI enhances performance in real-world collaboration tasks, underscoring the value of interpretability. HULA [64] is a human-in-the-loop software framework that enables practitioners to refine plans and code iteratively, aligning automation with developer expertise. Magentic-UI [49] is an open-source platform that offers mechanisms such as co-planning, co-tasking, and action approval, emphasizing human–agent interaction over task completion.

Motivated by these insights, we propose **HiLSVA** (Human-in-the-Loop Scientific Visualization Agentic System), designed to bridge the gap between autonomous visualization agents and the inherently human-centered goals of SciVis. HiLSVA is grounded in the principle that automation should amplify, rather than replace, human analytical capabilities [59]. By combining mixed-initiative interaction with robust agentic infrastructure, our system seeks to enhance both efficiency and transparency in human–AI collaboration for SciVis. Specifically, HiLSVA makes the following contributions:

Human-centered collaboration through oversight, provenance, and safe execution. Grounded in principles of mixed-initiative interaction [28], HiLSVA introduces fine-grained mechanisms that enable shared initiative between users and agents while preserving human authority throughout the SciVis workflow. The system supports *joint planning and execution*, where users and agents collaboratively construct explicit, stepwise plans that must be reviewed and approved before execution. During execution, HiLSVA treats the user as a *human-in-the-loop user proxy*, enabling seamless intervention through natural language or direct manipulation of visualization tools, with fluid handoff between human and agent control. It further provides *stepwise*

• *The authors are with the Department of Computer Science and Engineering, University of Notre Dame, Notre Dame, IN 46556, USA. E-mail: {kai, mdo23, chaoli.wang}@nd.edu.*

Manuscript received xx xxx. 202x; accepted xx xxx. 202x. Date of Publication xx xxx. 202x; date of current version xx xxx. 202x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.202x.xxxxxxx

control and provenance tracking, recording execution steps, software states, and visualization outputs to support undo, comparison of alternative trajectories, and reuse of validated workflows. To ensure reliability and safety, all agent actions are performed within *sandboxed execution environments* that enforce session isolation and require explicit user approval for critical or irreversible operations. In addition, *learn-at-test-time with human-in-the-loop feedback* enables HiLSVA to detect uncertainty, proactively query users for clarification, and incorporate feedback into a dynamic knowledge base that adapts across tasks.

Domain-adapted integration with SciVis pipelines. Rather than relying on slow, error-prone GUI-based control used by general-purpose agents [6, 82], HiLSVA leverages MCP to integrate directly with SciVis engines via specialized, tool-aware agents. It incorporates a ChatVis-style code agent [53] for robust generation and iterative refinement of visualization scripts, as well as a ParaView agent built on ParaView-MCP [43] for reliable, parameterized interaction with the visualization engine. These agents provide a modular integration layer that ensures efficient, reproducible execution and allows HiLSVA to adapt to diverse visualization pipelines without altering the overall system design.

Comprehensive evaluation across autonomy levels and participant expertise. We evaluate HiLSVA through selected case studies and a user study examining system behavior across different levels of user involvement. Twelve participants interacted with three autonomy modes in a controlled, balanced setup, ensuring that each participant experienced all modes across tasks, followed by a final task with free-mode selection. The results indicate a clear tradeoff, where higher automation improves execution efficiency, while increased human involvement enhances control and oversight. Our study further investigates how participants with varying levels of expertise, from novices to domain scientists and SciVis experts, interact with and benefit from the system. Across all expertise levels, users were able to complete non-trivial SciVis tasks with HiLSVA support. Post-study feedback indicates strong perceived usability, transparency, and support for iterative refinement, suggesting that HiLSVA effectively supports users with diverse backgrounds, including novices.

These contributions establish HiLSVA as a practical and extensible system for human-in-the-loop SciVis. By foregrounding user agency, adaptability, and transparency, HiLSVA advances the design of SciVis agents that empower people to derive meaningful insights and scientific analysis rather than being sidelined by automation.

2 Related Work

LLM agents have enabled new opportunities for SciVis through natural language interaction, workflow automation, and domain-specific analysis. Existing systems have explored autonomous SciVis pipelines, visualization-specific agents, and human-agent collaboration. As shown in Table 1, however, these capabilities are often studied in isolation. HiLSVA instead centers sustained human participation, combining domain-adapted SciVis agents with human oversight, mixed-initiative interaction, provenance-aware workflows, safe execution, and learn-at-test-time (LTT) adaptation from human feedback. These capabilities position agentic SciVis as a human-centered process in which autonomy supports, rather than replaces, human reasoning and exploration.

Table 1: Comparison of representative agentic systems.

method	SciVis adaptation	human oversight	mixed-initiative	workflow provenance	safe execution	LTT with human feedback
Magentic-UI [49]	✗	✓	✓	✗	✓	✗
Cocoa [22]	✗	✓	✓	✗	✗	✗
CowPilot [33]	✗	✓	✗	✗	✗	✗
VizGenie [11]	✓	✗	✗	✗	✗	✓
NLI4VolVis [5]	✓	✗	✓	✗	✗	✗
InferA [66]	✓	✗	✗	✓	✓	✗
SASAV [62]	✓	✗	✗	✗	✗	✗
AI VIS co-scientist [47]	✓	✓	✗	✓	✗	✗
HiLSVA (ours)	✓	✓	✓	✓	✓	✓

2.1 LLM-Based Agents

General-purpose agents. General-purpose LLM agents aim to operate across diverse environments with minimal task-specific engineering. Recent surveys identify reasoning, planning, tool use, memory, and reflection as core capabilities of such systems [45, 77]. Key advances include API and external-tool use [54, 56], interaction with operating systems and GUIs [76, 84], search-based reasoning with backtracking [36, 81], and self-reflection for iterative refinement [51]. These capabilities allow LLMs to move beyond single-step text generation toward long-horizon, tool-grounded problem solving. More recently, self-evolving agents have explored continual adaptation beyond static tool chains. For example, SEAgent [63] studies autonomous software learning through curriculum-driven exploration, while Fang et al. [21] survey broader mechanisms and challenges for lifelong agentic systems.

Visualization agents. Visualization agents adapt general agentic capabilities to tasks where visual feedback, iterative refinement, and domain-specific evaluation are central. MatPlotAgent [79] combines query understanding, code generation, and visual feedback for scientific plotting, while LIDA [16] employs a grammar-agnostic pipeline for data summarization, goal exploration, and visualization specification. AVA [44] and IntuiTF [73] further demonstrate how multimodal feedback can guide low-level visualization parameter optimization. Other systems emphasize collaborative, tool-grounded, or multi-agent workflows: ChatVis [53] iteratively generates and refines ParaView scripts through dialogue, CoDA [13] coordinates specialized agents for visualization specification and validation, and ParaView-MCP [43] connects LLMs to ParaView through MCP-based structured tool calls. Similar domain-specific systems have been developed for bioimage visualization and topological data analysis, such as BioImage-Agent [48] and TopoPilot [25]. Furthermore, SciVisAgentSkills [4] explores reusable agent skills for improving general-purpose coding agents across SciVis tools. Vonderhorst et al. [71] compared how different agent designs and interaction modalities affect multi-step SciVis task performance. While these approaches demonstrate the promise of LLMs for visualization creation, refinement, and automation, they primarily focus on agent capability, tool integration, or output generation rather than sustained human oversight throughout SciVis workflows.

2.2 Human–Agent Collaboration

Reintroducing humans into the decision loop challenges the pursuit of full automation, often introducing additional effort and coordination costs. However, empirical evidence suggests that pairing humans with agents can yield significant productivity gains compared to human-only workflows. For example, randomized controlled trials of GitHub Copilot have shown that developers complete programming tasks over 50% faster with AI assistance [52]. Meanwhile, studies warn that improvements in an AI system's standalone accuracy do not always translate into better team performance: model updates can break established user expectations, leading to incompatibility between human mental models and machine behavior [7, 9]. Communication remains another open challenge, as humans and agents must align on intent, interpretability, and appropriate levels of human oversight to collaborate effectively [8].

Multi-agent collaboration. Multi-agent systems provide one route for scaling coordination. Classical work emphasized robustness, modularity, and division of labor as the main benefits of distributed agents [61, 67]. Modern approaches bring these ideas into the LLM era, enabling specialized agents to converse, plan, and solve problems collectively. Frameworks such as AutoGen [75] exemplify this paradigm, while recent surveys synthesize emerging methods for communication, specialization, and joint problem solving, along with open challenges in evaluation and safety [26, 65].

Mixed-initiative systems. Beyond coordination among artificial agents, an equally important strand of research emphasizes the role of humans as integral members of the team. Horvitz's principles for mixed-initiative interfaces [28] advocate balancing automation with user control, supporting uncertainty handling, dialogue, and smooth handoffs between humans and agents. Shneiderman and Maes [59] emphasized that the central design tension is not between automation and control, but rather how to delegate effectively while preserving clarity, responsibility, and human judgment. Empirical work further demonstrates that interpretability enhances collaboration: explainable AI systems with visual explanations enable users to better understand and oversee model behavior, thereby improving joint task performance compared to black-box models [57]. In visualization specifically, Dhanoa et al. [15] argued that full automation is neither realistic nor desirable;

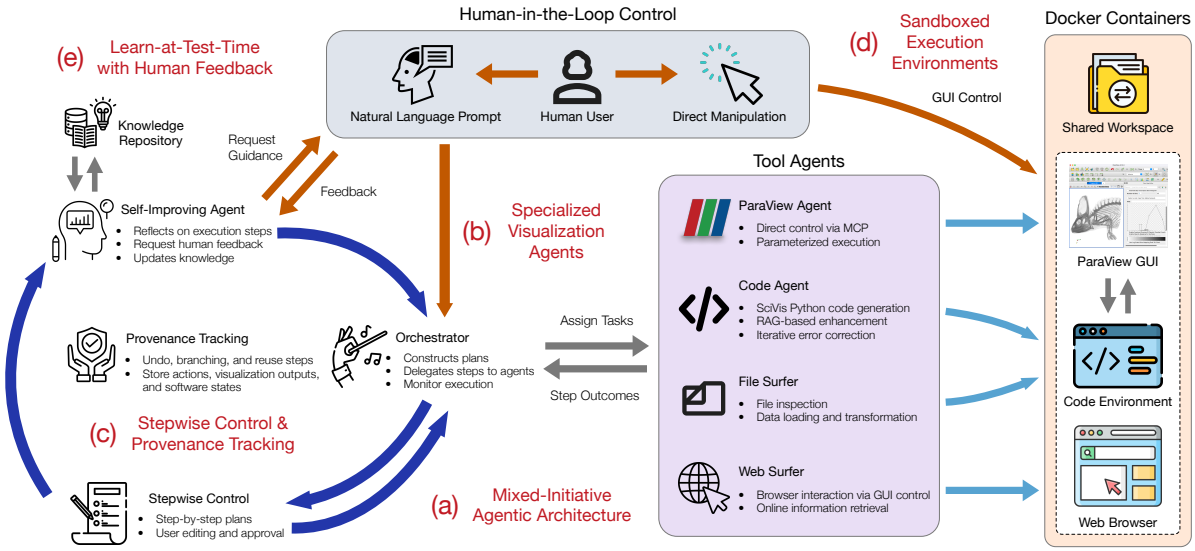

Figure 1: Overview of HiLSVA. (a) An orchestrator interprets user intent, constructs explicit stepwise plans, and coordinates specialized agents with bidirectional human–agent interaction. (b) Specialized visualization agents execute SciVis actions via direct API calls or GUI control and report execution outcomes to the orchestrator. (c) Workflows are represented as editable steps, planned and executed actions, software states, and visualization outputs, with these recorded to support undo, branching, and reuse. (d) Tool interactions run in isolated Docker containers, enabling safe, parallel execution across sessions. (e) A self-improving agent reflects on execution outcomes, assesses uncertainty, queries users when needed, and updates a knowledge repository during inference.

visualization remains fundamentally human-centered, designed to amplify reasoning and support open-ended sensemaking rather than to replace human agency.

LLM-based collaboration systems. A growing line of work explores how LLM agents can share initiative with humans in open-ended tasks. Cocoa [22] introduces a co-planning interface in which both the human and the agent decide on task decomposition upfront; however, the collaboration remains static, as dynamic handoffs or replanning are not supported. CowPilot [33] focuses on web navigation, providing pause, resume, and override capabilities that let humans recover from agent errors, yet lacks structured co-tasking or proactive negotiation beyond corrections. HULA [64] targets human-in-the-loop software development, demonstrating the benefits of pairing LLM agents with developers for code refinement, but its scope is restricted to a narrow task domain. Magentic-UI [49] advances the state of the art with fine-grained, mixed-initiative interaction: humans and agents can dynamically exchange control of subtasks, replan mid-execution, and jointly approve critical actions.

Human–agent collaboration in SciVis. SciVis poses unique challenges for collaboration, as tasks are open-ended, visually grounded, and difficult to specify completely in advance. Recent agentic SciVis systems have explored different ways to reduce this interaction burden. VizGenie [11] dynamically generates and validates visualization scripts to extend system functionality, while NLI4VolVis [5] supports natural language interaction, open-vocabulary querying, and real-time editing for volume visualization. TexGS-VolVis [68] further enables expressive scene editing through textured Gaussian splatting. Zhang et al. [83] study semantic alignment for natural-language-driven flow visualization. InferA [66] supports multi-agent exploration of cosmological ensemble data. More recent systems also push toward increasingly autonomous workflows: SASAV [62] performs self-directed scientific analysis and visualization through automated data profiling, knowledge retrieval, and parameter exploration for visualization, while AI VIS co-scientist [47] studies an end-to-end agent harness for generating visual analysis applications from high-level task descriptions. These systems demonstrate the growing capabilities of LLM-based agents for SciVis, but primarily emphasize natural-language interaction, autonomous execution, or domain-specific analysis.

2.3 Evaluating Human–Agent Collaboration

Measuring the effectiveness of human–agent teams requires going beyond outcome metrics to capture process, interaction quality, and consistency. Collaborative Gym [58] provides a modular environment for benchmarking human-agent teaming, while τ-bench [80] evaluates tool-augmented agents through multi-turn interactions with humans and APIs under domain-specific constraints. Fragiadakis et al. [23] further argued that evaluation should integrate both quantitative and qualitative measures to assess interpretability, transparency, and the quality of collaboration. Extending this perspective to visualization, Ai et al. [2] emphasized systematic evaluation of agentic SciVis systems, and Do et al. [17, 18] further highlighted the importance of rigorously assessing human and multimodal LLMs' understanding and interpretation capabilities in SciVis.

Building on these insights and the human-centered nature of SciVis workflows, we evaluate HiLSVA along three axes: (1) autonomous vs. human-in-the-loop operation to assess mixed-initiative benefits, (2) the effect of user expertise on interaction patterns and outcomes, and (3) case studies that illustrate system behavior in realistic SciVis scenarios. Unlike fully automated benchmarks such as NL2SciVis [46] and SciVisAgentBench [3], human-in-the-loop studies inherently involve participants with different backgrounds and expertise levels, making large-scale reproducible benchmark-style evaluation difficult. Therefore, we focus on controlled user studies and qualitative analyses that capture the collaborative nature of human–agent SciVis workflows.

3 Our Approach

We implement HiLSVA on top of Magentic-UI [49], which provides the general mixed-initiative infrastructure used for planning, execution, and action approval. As illustrated in Figure 1, HiLSVA extends this foundation with SciVis-specific capabilities, including specific agent designs, provenance-aware execution with human oversight, multimodal visualization control via MCP tool calls, generated scripts, and direct user interaction, sandboxed environments, and test-time adaptation via human feedback. This section details these design choices across mixed-initiative agentic architecture, mechanisms for human oversight and safe execution, LTT adaptation, and interactive interface design. Although the current implementation centers on ParaView, the architecture is designed to support multiple SciVis backends. In Figure 2, we highlight the key mixed-initiative capabilities of HiLSVA.

3.1 Mixed-Initiative Agentic Architecture

Multi-agentic system design and orchestration. HiLSVA adopts a multi-agent architecture centered around a lead **orchestrator** that coordinates a set of specialized sub-agents. These include a **web surfer**

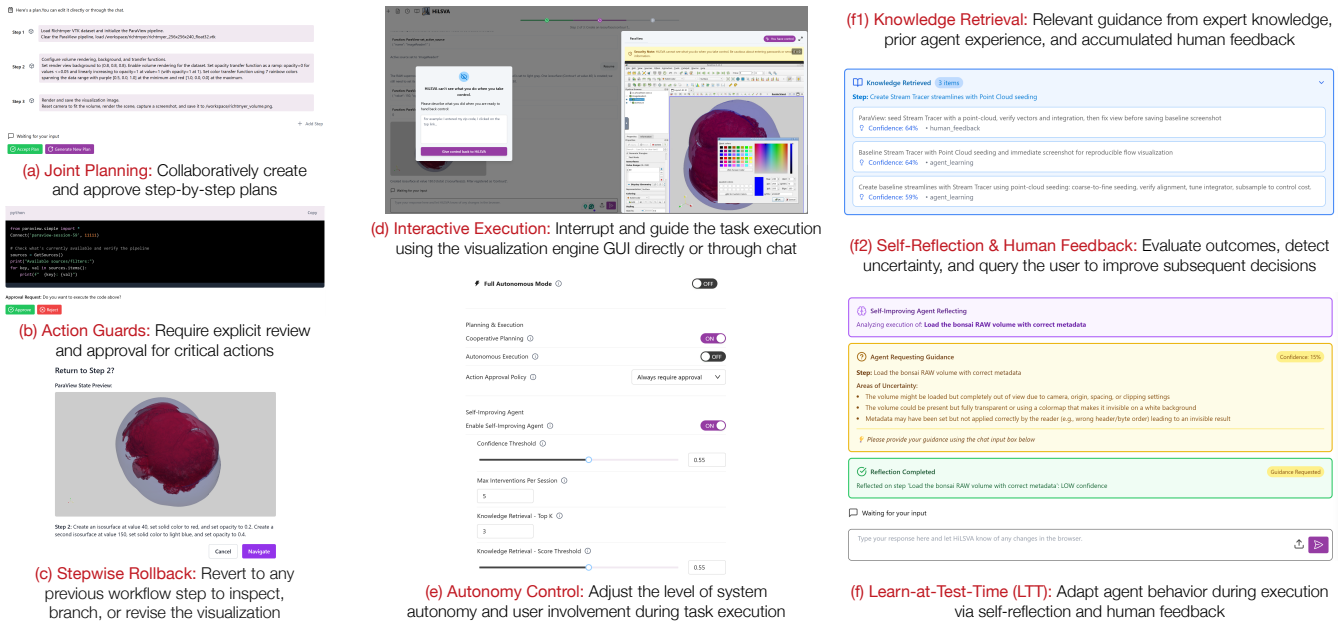

Figure 2: Key mixed-initiative capabilities of HiLSVA. The system supports (a) collaborative planning, (b) guarded execution, (c) stepwise rollback, (d) interactive user steering, (e) autonomy control, and (f) LTT adaptation through retrieval, reflection, and human feedback.

for browser-based information seeking, a **code agent** for generating and executing Python and shell scripts to manipulate SciVis tools, a **file surfer** for file inspection and transformation, a **ParaView agent** that directly controls the ParaView GUI via an MCP server, and a **self-improving agent** that supports test-time learning through human feedback. These agents were selected to support backend-agnostic SciVis workflows. The orchestrator interprets user intent, constructs an explicit plan, assigns each step to an appropriate agent, tracks execution progress, and generates the final response to the user. The ParaView agent and code agent provide standard interfaces to visualization software, enabling the architecture to be extended to different backends, while the remaining agents extend the system with external information access and experience-driven adaptation.

The system follows a plan-first workflow. Upon receiving a task, the orchestrator synthesizes a stepwise plan, with each step assigned to a specific agent. Rather than relying on multi-turn dialogue for refinement, HiLSVA exposes the plan as an editable artifact. Users may reorder steps, add or remove actions, or modify step instructions before execution, enabling precise and low-effort control over the planning stage (Figure 2(a)). Execution begins only after the user approves the plan. Once execution starts, HiLSVA operates in a mixed-initiative manner. At each step, the orchestrator monitors execution status, determines whether the step has been completed, and may revise instructions or trigger replanning when necessary. Before executing a step, the orchestrator may invoke the self-improving agent to retrieve relevant domain knowledge from an external knowledge base. After each step, the self-improving agent can reflect on outcomes and, when appropriate, solicit additional human guidance to refine subsequent actions.

Specialized visualization agents. In our current implementation, HiLSVA runs ParaView [1] inside isolated Docker environments, ensuring reproducible and safe execution. We selected ParaView because it is one of the most comprehensive and widely adopted SciVis platforms, providing a rich set of visualization and analysis capabilities while uniquely supporting multiple interaction modalities on the same visualization instance, including MCP tool calls, generated Python scripts, and direct GUI manipulation by the user, enabling seamless handoff between human and agent control.

The *code agent* in HiLSVA adopts techniques similar to ChatVis [53], including structured prompt decomposition, retrieval-augmented generation (RAG) over documentation and code examples, and iterative error checking with feedback-driven refinement until executable visualization scripts are produced. This design enables robust generation and correction of SciVis code scripts that involve complex tool APIs. To support reliable tool interaction, the *ParaView agent* interfaces with the visualization engine via ParaView-MCP [43]. The MCP server provides a higher-level, function-oriented abstraction over ParaView's Python API, enabling the agent to invoke visualization operations via parameterized function calls rather than low-level scripting.

The architecture itself is backend-agnostic. Visualization tools are encapsulated within sandboxed containers and accessed through specialized agents. Consequently, other domain-specific visualization environments can be integrated without changing the overall system design. Examples include VMD [32] for molecular visualization, napari [60] for bioimage visualization, and TTK [70] for topology analysis. If corresponding MCP servers or tool interfaces (e.g., GMX-VMD-MCP [20], BioImage-Agent [48], TopoPilot [25]) are available, they can be incorporated as additional specialized agents within the existing framework.

3.2 Human Oversight, Provenance, and Safe Execution

Human-in-the-loop user proxy. The user is treated as a first-class participant in the multi-agent team, and the orchestrator may delegate specific steps to the user as part of the shared execution process. During execution, HiLSVA supports mixed-initiative co-execution [22, 33, 49]. Users may intervene at any time through natural-language instructions or direct interaction with the visualization environment, such as adjusting transfer functions or camera parameters when visual intent is difficult to specify in words. After intervention, users can return control to the agent, enabling fluid handoff between manual and automated execution (Figure 2(d)). Initiative also flows in the opposite direction: when the orchestrator or a specialized agent detects ambiguity, deviation from the approved plan, or insufficient confidence to proceed, the system proactively queries the user for clarification. Moreover, HiLSVA enforces explicit user approval for critical or potentially irreversible actions, such as executing generated code or sensitive SciVis tool calls (Figure 2(b)). Users can also adjust the level of system autonomy, balancing agent autonomy and human intervention (Figure 2(e)).

Stepwise control and provenance tracking. Reproducibility and accountability are fundamental requirements in scientific research and are closely tied to provenance [24]. Provenance provides a structured record of how results are produced, enabling interpretation, verification, and reuse of computational workflows. Prior work in visualization and data analysis has demonstrated the value of provenance for supporting sensemaking, reflection, and responsible system behavior [10, 50, 74].

HiLSVA provides fine-grained, stepwise control through an explicit workflow representation, allowing users to revisit and revise their anal-

ysis as it unfolds. For each execution step, the system records both *prospective* provenance (the planned sequence of actions) and *retrospective* provenance (the executed actions, software states, and visualization outputs). Users can select any prior step to restore the corresponding state, modify the remaining plan from there, and continue execution along the updated workflow plan (Figure 2(c)). In addition to tracking individual executions, HiLSVA supports storing and reusing complete workflow plans. After a task is completed, users may choose to save the validated plan along with its associated execution provenance. Saved plans can be replayed on the original task, used as guidance for new tasks, or automatically retrieved by the orchestrator.

Unlike plan-level retrieval, this workflow reuse is stateful. Users can restore an arbitrary step within a saved workflow together with the corresponding visualization software state, inspect intermediate results, branch from that point, and continue execution with revised instructions. This enables provenance-aware reuse and refinement of entire SciVis workflows rather than only reusing planning artifacts.

Sandboxed execution environments. HiLSVA employs plug-and-play sandboxed execution to ensure safety and reproducibility. Each dialogue session runs within its own isolated Docker environment for the visualization engine (i.e., ParaView), code execution, web browsing, and file access. These containers share a controlled workspace mapped to the local file system, enabling coordinated interaction among agents while preventing unintended side effects across sessions. HiLSVA also supports parallel execution of multiple tasks, allowing users to safely experiment with different settings without compromising the host system. Users can switch between active sessions, each backed by an independent ParaView Docker instance, with a session status indicator that indicates whether user input is required.

3.3 LTT with Human-in-the-Loop Feedback

LTT refers to a model's ability to adapt its behavior during inference after deployment. For LLMs, common LTT mechanisms include in-context or few-shot learning [12, 19], RAG [39], and, more recently, test-time fine-tuning [31]. In the context of agentic systems, self-learning agents aim to improve autonomously through interaction with their environment [21, 63]. HiLSVA builds on this line of work by explicitly incorporating humans into the test-time learning loop [27], enabling agents to adapt through structured self-reflection and targeted human feedback during task execution (Figure 2(f)).

Problem formulation. HiLSVA follows a plan-first-then-execute workflow. For a given task, the agent team executes a sequence of steps $\mathscr{S} = (s_1, s_2, \ldots, s_N)$, where each step corresponds to a dynamically evolving environment $\mathscr{E} = (e_1, e_2, \ldots, e_N)$. To complete step s_i, the agents perform a sequence of actions $a_i \in \mathscr{A}$. The environment state e_i includes the visualization engine state, auxiliary tool states, intermediate files, and relevant knowledge retrieved from an external repository. To operate effectively in this changing environment, the agent team maintains an internal state Θ and receives guidance from a human user $\mathscr{H}$ within a predefined interaction budget B.

LTT. During execution of step s_i, the agent team maintains an internal state $\theta_i \in \Theta$ and follows a decision policy $\pi : \mathscr{E} \times \Theta \to \mathscr{A}$, which produces actions $a_i = \pi(e_i, \theta_i)$. We model LTT as an update of the internal state after each step $\theta_{i+1} = f(\theta_i, e_i, a_i, r_i; q_i, h_i)$, where r_i denotes the agent's self-reflection after completing step s_i, q_i is a clarification query posed to the user, and h_i is the corresponding human feedback.

Knowledge updating with human feedback. In HiLSVA, the internal state of the agent team is instantiated as a vector-based knowledge repository R_i, i.e., $\theta_i \approx R_i$. Each knowledge item is represented by a natural language caption and indexed using a BERT-based embedding of that caption. The initial repository R_1 is seeded with general SciVis knowledge, and is incrementally augmented with insights derived from prior task executions and human feedback.

Before executing step s_i, relevant knowledge items are retrieved from R_i and incorporated into the environment state e_i to guide agent actions. Retrieval is based on a composite score defined as $\text{score} = w_{\text{valid}} \cdot s_{\text{semantic}} \cdot s_{\text{confidence}} \cdot s_{\text{recency}}$, where $w_{\text{valid}} \in \{0, 1\}$ denotes a validity weight indicating whether an item is considered outdated, s_{semantic} is the cosine similarity between the BERT embedding of the step description for s_i and the embedding of the knowledge caption, and $s_{\text{confidence}} \in [0, 1]$ reflects the reliability of the knowledge item. Knowledge derived from human feedback is assigned a confidence score of 1, while insights obtained through self-reflection receive confidence scores graded by the self-improving agent based on its assessment of action reliability and execution outcomes. The recency score is defined as an exponential decay, $s_{\text{recency}} = \exp(-\lambda \cdot \Delta t)$, where Δt is the normalized time since the item's last validation and λ is a decay rate. Certain foundational knowledge items (e.g., core SciVis principles) are marked as non-expiring and assigned a recency score of 1.

After executing step s_i, the self-improving agent performs a self-reflection to assess execution quality, assigns a confidence score, and summarizes any ambiguities requiring clarification as a query q_i. Each query incurs a cost $c(q_i) > 0$, set uniformly to 1 in our experiments for simplicity. In principle, different query types could be assigned different costs depending on their cognitive or interaction burden. If the confidence score falls below a predefined threshold and the accumulated cost satisfies $\sum_{j=1}^{i} c(q_j) \le B$, the agent requests human feedback $h_i = \mathscr{H}(e_i, a_i, q_i)$. The knowledge repository is then updated according to $R_{i+1} = g(R_i, r_i, q_i, h_i)$, where the update function g constitutes the core of the LTT function f. g stores both self-reflection summaries and human feedback as new knowledge items, along with associated metadata including timestamps, confidence scores, and LLM-generated captions. These metadata are subsequently used to support informed retrieval in future steps.

Self-improving agent and user expertise. LTT with human feedback in HiLSVA is implemented via retrieval-based adaptation rather than model weight updates or architectural changes. A dedicated self-improving agent observes all actions and environment states, performs confidence assessment and self-reflection after each execution step, and decides whether to request human input. Validated insights and self-reflections are stored as natural-language knowledge items associated with confidence and recency scores. These knowledge items are retrieved to enable adaptation through accumulated experience and human feedback. This mechanism can be disabled to support ablation studies that quantify its impact on task performance. HiLSVA is designed to collaborate with users of varying expertise levels, including non-experts, domain experts, and SciVis experts; accordingly, our evaluation examines how user expertise influences interaction patterns, collaboration dynamics, and outcome quality.

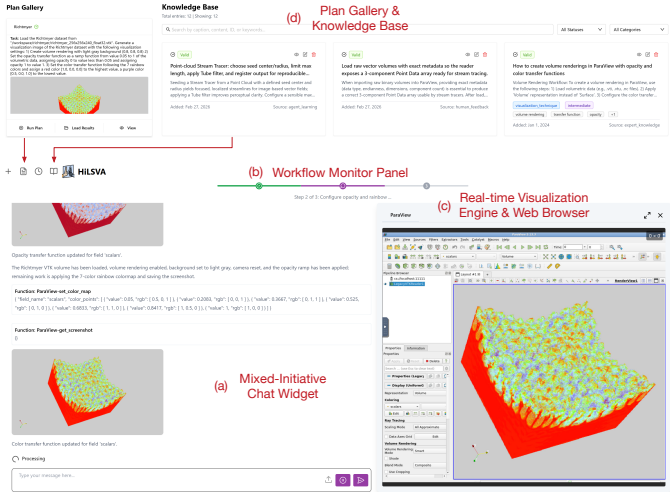

Figure 3: Overview of the HiLSVA interface. (a) *Mixed-initiative chat widget* for natural language planning, step refinement, and uncertainty prompts. (b) *Workflow monitor panel* that displays execution progress and step status, and supports clickable state transitions for returning to prior steps. (c) *Real-time visualization engine* and *web browser*, where the agent's actions and user interventions directly operate on the ParaView rendering environment. (d) *Plan gallery* and *knowledge base* that accumulate validated workflows and learned experience to support provenance-aware reuse and LTT adaptation.

3.4 Interactive Interface

HiLSVA provides an interface that tightly couples planning, execution, and visualization. The interface consists of three primary components: a *mixed-initiative chat widget* for natural-language interaction, a *workflow monitor panel* for tracking execution progress and provenance, and a *real-time visualization engine* and a *web browser*. As shown in Figure 3, these components operate in synchrony to support collaborative SciVis workflows. The mixed-initiative chat widget serves as the primary interaction channel for user–agent interaction. Through chat, users can specify tasks, inspect and edit execution plans, approve or reject actions, review generated code, and provide feedback in response to agent self-reflection. The workflow monitor panel exposes the current execution state as a stepwise process. Users can inspect progress, undo completed steps, and return to prior states using the recorded provenance. In parallel, the visualization engine and the web browser update in real time as agents interact with external tools. Users may intervene directly by taking control of the GUI at any point, then seamlessly resume automated execution. In addition to these core components, HiLSVA includes a *plan gallery* and a *knowledge base* that present previously saved workflows, accumulated knowledge, and their associated provenance, as well as a *session monitor sidebar* that allows users to manage multiple concurrent task sessions and view their execution status.

4 Results and Evaluation

This section presents our approach to analyzing and evaluating HiLSVA. We combine structured case studies, a controlled user study followed by surveys and interviews, and multimodal analysis integrating benchmark-driven quantitative metrics and qualitative evaluations. Together, these methods enable a comprehensive investigation of how mixed-initiative agentic systems can support data exploration, visualization construction, and scientific analysis. For the backbone LLM, we adopt GPT-5.2 for the orchestrator and Claude-Sonnet-4.6 for the other tool agents.

The Institutional Review Board at our University approved the user study conducted in this work. Before accessing the study materials, all participants reviewed and signed an informed consent form. They received an explanation of the study's objectives and procedures, and consent was documented by completing a hard-copy consent form.

4.1 SciVis Stages and Case Study Design

Our evaluation is structured around three canonical SciVis stages: *data exploration*, *data visualization*, and *scientific insight*, which guide us in designing case studies of increasing analytic complexity. In the **data exploration** stage, users familiarize themselves with unfamiliar multidimensional datasets by selecting visualization techniques, identifying salient structures, and determining how to begin analysis. This aligns with our **basic action tasks**, which test HiLSVA's reliability in executing foundational operations such as loading data, adjusting views, filtering, and generating renderings. The **data visualization** stage focuses on developing iterative, mixed-initiative workflows to address analytical questions. Here, both user and agent collaborate through planning, refinement, and clarification. This corresponds to our **visualization workflow tasks**, which require multi-step reasoning, coordinated pipeline construction, and negotiation to isolate regions of interest, refine transfer functions, or generate comparative views. Finally, the **scientific insight** stage focuses on moving beyond direct visualization of raw data to derive, compare, and interpret scientifically meaningful features. This parallels our **scientific analysis tasks**, which involve analysis-driven, open-ended objectives such as computing derived quantities, identifying structural patterns, and extending workflows across different simulation conditions. HiLSVA's adaptive behavior and the ability to reuse previously learned workflows are particularly critical at this stage. By aligning evaluation stages with progressively complex task classes, we systematically examine when and how human–agent collaboration enhances analytic reasoning.

4.2 Case Studies

We present five case studies of varying complexities to showcase HiLSVA's capabilities. These include two basic action tasks (foot and hurricane), two visualization workflow tasks (tornado and combustion), and one scientific analysis task (half-cylinder). For brevity, the case studies begin after a visualization strategy has been selected. In practice, HiLSVA supports co-planning to help users identify suitable visualization and analysis techniques from high-level goals.

Foot. The dataset is a CT scan of a human foot with a resolution of 125×255×183. We use this example to illustrate a basic action task. As shown in Figure 4, the user first requests an initial isosurface visualization to reveal bone structures while distinguishing bone from surrounding skin and soft tissue. The HiLSVA orchestrator generates a plan that loads the dataset, extracts isosurfaces that separate bone from surrounding tissue, and assigns colors to highlight anatomical structures. After the plan is approved, the ParaView agent executes these steps and produces an initial isosurface visualization.

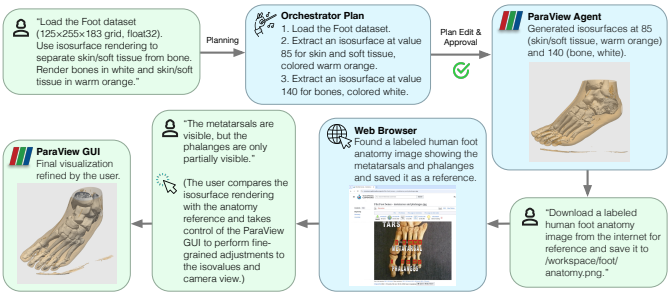

Figure 4: Case study of the foot (basic action task). HiLSVA generates an initial isosurface visualization, retrieves an anatomy reference from the web, and supports user-driven refinement through direct GUI interaction.

Next, the user asks HiLSVA to download a labeled anatomy image of the human foot showing the metatarsals and phalanges. The web surfer retrieves this reference from the internet, providing anatomical context that is not directly available from the raw visualization. By comparing the current isosurface with the anatomical reference, the user observes that the metatarsals are clearly visible, whereas the phalanges are only partially visible. Further refinement through natural language alone would require specifying detailed parameters, such as precise isovalues or camera adjustments, which can be cumbersome. Instead, HiLSVA's mixed-initiative interaction allows the user to directly control the ParaView GUI and make fine-grained adjustments to the visualization. Through this interactive refinement, the user quickly improves the isosurface rendering and makes the anatomical structures more clearly visible. This example demonstrates how HiLSVA supports data exploration through iterative visualization refinement and contextual information retrieval.

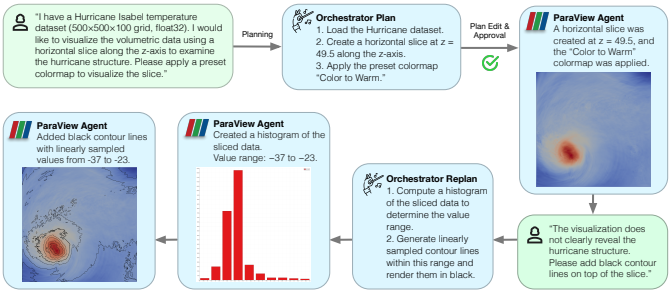

Figure 5: Case study of the hurricane (basic action task). HiLSVA first constructs a horizontal slice of the temperature field, then analyzes the data distribution and overlays contour lines to better reveal the temperature distribution.

Hurricane. The dataset is an atmospheric simulation of Hurricane Isabel with a resolution of 500×500×100. In this example, we use the temperature field at a single time step to examine the hurricane structure. We use this example to demonstrate a basic action task supported by HiLSVA. As shown in Figure 5, the user first requests a visualization of the hurricane by creating a horizontal slice along the z-axis and applying a preset colormap. The HiLSVA orchestrator generates a plan

that loads the dataset, extracts a horizontal slice at $z = 49.5$, and applies the preset Color to Warm colormap. After plan approval, the ParaView agent executes these steps and produces an initial slice visualization.

However, the resulting image does not clearly reveal the hurricane structure. The user therefore asks the system to overlay contour lines on the slice to highlight the temperature distribution better. In response, the orchestrator first computes a histogram of the sliced data to determine the value range, then generates contour lines using linearly sampled values within this range. The ParaView agent executes these steps and overlays black contour lines on the slice, making the hurricane's structure and temperature gradients more clearly visible. This example illustrates how HiLSVA supports exploratory analysis by refining visual representations to reveal salient temperature structures.

Tornado. The dataset is a vector field representing a simulated tornado, with a resolution of $64\times64\times64$. We use it to show a visualization workflow task supported by HiLSVA. As shown in Figure 6, the user first specifies basic requirements and asks the system to load the dataset. The user then requests a streamline visualization using ParaView's Stream Tracer filter with a Point Cloud seeding strategy. However, the user is uncertain about the optimal parameter configuration, including the seeding sphere center, radius, and maximum streamline length, and therefore wishes to explore alternative settings.

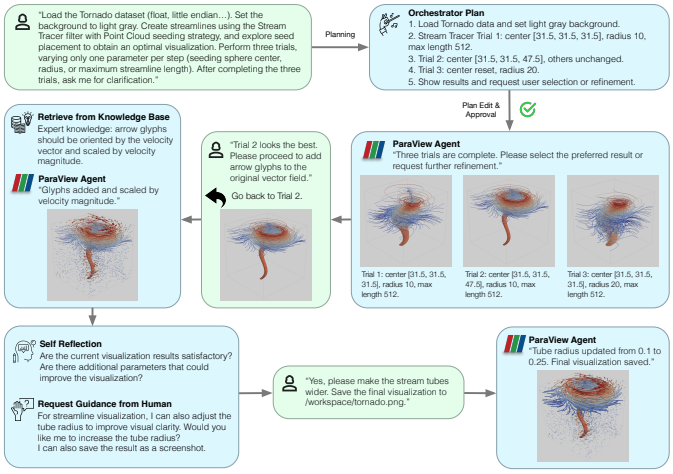

Figure 6: Case study of the tornado (visualization workflow task). HiLSVA explores parameters for visualizing streamlines, supports rolling back to the selected trial, and adds knowledge-guided glyph visualization.

In response, the HiLSVA orchestrator generates a five-step plan. The workflow begins by loading the dataset and setting the background color. The system then performs three parameter trials. Trial 1 places the seeding sphere at the volume's center [31.5, 31.5, 31.5] (the data extent is [0, 63]) with radius 10 and maximum streamline length 512. Trial 2 shifts the seeding sphere's center to [31.5, 31.5, 47.5] while keeping other parameters fixed. Trial 3 moves the seeding sphere's center back to the volume's center and increases its radius to 20. HiLSVA presents the resulting visualizations and prompts the user to select the preferred configuration or request further refinement.

After inspecting the results, the user identifies Trial 2 as optimal and requires no further tuning. Using HiLSVA's provenance-aware workflow monitor, the user can instantly restore the ParaView state corresponding to Trial 2 with a single click. All parameters and rendering states are faithfully recovered. Building on the selected streamline visualization, the user next requests a Glyph filter on the original vector field using arrow glyphs. The self-improving agent consults the knowledge repository and retrieves expert guidance indicating that glyphs should be oriented by the velocity vector and scaled by velocity magnitude. Guided by this knowledge, the orchestrator generates a new plan and instructs the ParaView agent to produce colored arrow glyphs scaled by velocity magnitude using the same color map.

After generating the glyph visualization, the self-improving agent performs a self-reflection step to evaluate the results and identify parameters that could further improve clarity. It suggests adjusting the streamline tube radius and asks the user for confirmation. The user agrees and requests wider stream tubes. The ParaView agent then applies a Tube filter and increases the tube radius from 0.1 to 0.25, and saves the final visualization.

Finally, the user may optionally preserve the workflow for future reuse. HiLSVA stores not only the stepwise plan but also the full provenance record, including ParaView states and intermediate visualization snapshots. These artifacts can be reloaded in later sessions or referred to in other sessions. This case demonstrates how mixed-initiative interaction helps users evaluate alternatives and refine visualization designs.

Combustion. The dataset is a multivariate, time-varying simulation of turbulent reacting flow. In this example, we use two scalar variables: `mixfrac`, representing the mixture fraction of fuel and oxidizer, and `Y_OH`, the mass fraction of the hydroxyl radical, a commonly used indicator of reaction zones in combustion. We use a downsampled version of the dataset containing 33 timesteps, with a resolution of $240\times360\times60$. As shown in Figure 7, the user first requests visualization of the time-varying `Y_OH` variable and asks the system to export the animation as a video. The HiLSVA orchestrator generates a plan comprising three steps: determining the number of timesteps in the dataset, loading the time-varying `Y_OH` field, and performing volume rendering using the previously used colormap before exporting the animation. To determine the number of timesteps, the file surfer inspects the directory structure and identifies 33 raw data files corresponding to individual timesteps.

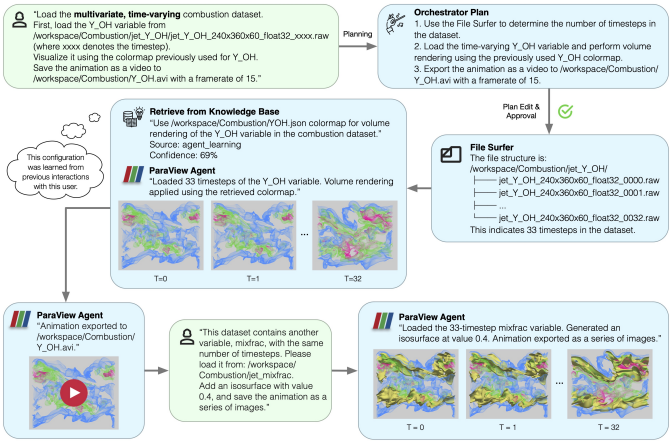

Figure 7: Case study of the combustion (visualization workflow task). HiLSVA visualizes the time-varying `Y_OH` scalar field using volume rendering, exports an animation, and then loads the `mixfrac` scalar field to generate an isosurface visualization across all timesteps.

Before visualization, the self-improving agent retrieves a previously learned configuration from the knowledge repository, suggesting the use of a dedicated colormap for volume rendering. This configuration was learned from prior interactions with the user and stored as reusable knowledge, enabling HiLSVA to adapt visualization parameters to user-specific preferences. Guided by this retrieved knowledge, the ParaView agent loads all timesteps of the `Y_OH` field and performs volume rendering across the sequence. The resulting frames are then exported as a video at the requested frame rate.

After the animation is generated, the user requests further analysis using another variable in the dataset, `mixfrac`, which shares the same temporal resolution. The user asks the system to load the time-varying field and extract an isosurface with an isovalue of 0.4. The ParaView agent loads the 33 `mixfrac` timesteps, generates an isosurface visualization for each, and exports the animation as an image sequence. This example highlights how HiLSVA supports iterative workflow refinement for multivariate, time-varying datasets.

Half-cylinder. The dataset is a small ensemble of numerical simulations of incompressible 3D flow around a half-cylinder. Each ensemble member corresponds to a different Reynolds number, and the simulations were resampled from adaptive unstructured grids onto a regular grid. In this example, we use a single timestep from the Reynolds

number $Re = 640$ case on a regular grid of $640\times240\times80$. We use this example to demonstrate a scientific analysis task supported by HiLSVA. Unlike the previous workflow-oriented examples, this case focuses on analysis-driven exploration. Users are not required to know specific vortex-identification techniques, such as the Q-criterion or λ_2 in advance. Instead, users may begin with high-level goals, such as exploring flow structures or identifying regions of complex behavior. Through co-planning and iterative dialogue, HiLSVA can propose candidate analysis strategies, explain their purpose, and execute the corresponding workflow. Domain experts may choose to specify particular analysis methods directly, which can further accelerate the process, but such knowledge is not required to initiate the analysis.

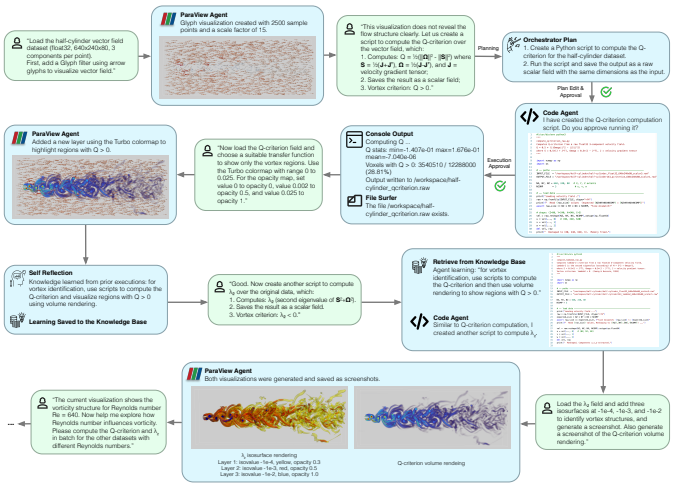

Figure 8: Case study of the half-cylinder (scientific analysis task). HiLSVA begins with a glyph-based visualization of the vector field, then computes and visualizes the Q-criterion and λ_2 fields to reveal vortex structures. The system further saves this successful workflow as reusable knowledge and supports extending the analysis to additional Reynolds numbers.

As shown in Figure 8, the user first requests a glyph-based visualization of the vector field using arrow glyphs. The ParaView agent then loads the dataset and generates the initial glyph visualization. However, the resulting visualization does not clearly reveal the underlying flow structures. Rather than stopping at this initial rendering, the user asks the system to help identify vortex regions through more scientifically meaningful analysis. This shift from direct depiction to derived-field analysis exemplifies a characteristic of scientific analysis tasks: the goal is not only to visualize the data but also to uncover latent structures that support interpretation. In response, HiLSVA generates a plan to compute the Q-criterion, asks for approval before executing the generated script, and saves the resulting scalar field. The script computes $Q = \frac{1}{2}\left(\|\mathbf{\Omega}\|^2 - \|\mathbf{S}\|^2\right)$, where $\mathbf{S} = \frac{1}{2}(\mathbf{J}+\mathbf{J}^\top)$ and $\mathbf{\Omega} = \frac{1}{2}(\mathbf{J}-\mathbf{J}^\top)$ are the symmetric and antisymmetric parts of the velocity gradient tensor $\mathbf{J}$. The ParaView agent then loads the computed Q-criterion field and applies volume rendering with a user-specified transfer function to highlight regions with $Q > 0$, revealing coherent vortex structures much more clearly than the initial glyph rendering.

Next, the user asks HiLSVA to analyze the flow further using the λ_2 method, another vortex identification measure. Similar to the previous step, the system generates and executes a second script to compute λ_2 from the original vector field, saves the result as a scalar field, and loads it into ParaView. The ParaView agent then creates three isosurfaces at different negative isovalues to visualize vortex structures at multiple strengths. Compared with the Q-criterion volume rendering, the λ_2 visualization offers an alternative perspective on the organization and relative intensity of vortical flow features. Together, these two visualizations of derived fields provide complementary evidence for interpreting the wake structure behind the half-cylinder.

This case study also highlights HiLSVA's mixed-initiative and self-improving capabilities during scientific analysis. After computing the Q-criterion and visualizing regions with $Q > 0$, HiLSVA summarizes this successful workflow through self-reflection and stores it as reusable knowledge. When the user later requests the λ_2 analysis, the system retrieves this prior experience on vortex identification to guide the subsequent workflow, reducing the need for the user to specify each intermediate step manually. After both visualizations are generated, the user can compare their results and decide how to proceed.

Finally, after obtaining the Q-criterion and λ_2 visualizations for the $Re = 640$ case, the user asks the system to extend the same analysis to the other data with different Reynolds numbers. HiLSVA can therefore apply these computations in batch, although we do not show the additional results here due to space limitations. Overall, this example illustrates how HiLSVA supports scientific analysis by going beyond direct visualization of raw data and enabling users to derive, validate, and compare scientifically meaningful flow features. As a mixed-initiative system, HiLSVA supports human-guided scientific analysis rather than autonomous scientific discovery. Assessing whether such workflows lead to discoveries requires longitudinal studies with domain scientists and remains an important direction for future work.

4.3 User Study

Experiment setup. We conducted a controlled user study with 12 participants under the University's IRB protocol. To examine how HiLSVA supports users with different mental models and prior experience, we recruited participants across three expertise levels: three SciVis experts, four domain scientists with familiarity with scientific datasets but limited experience with visualization tools, and five novices with neither domain nor visualization expertise. Each participant used HiLSVA on a workstation equipped with an NVIDIA RTX 5090 GPU and two 32-inch displays at 3840×2160 resolution. All participants completed the full study and received $30 compensation. Two authors administered the study sessions, while all tasks were completed independently by the participants. Throughout the study, HiLSVA logged user-agent interactions and workflow plans and generated visualizations for subsequent quantitative and qualitative analyses.

To evaluate the effect of human involvement, we studied three autonomy configurations with increasing degrees of mixed initiative: (1) full-autonomous mode, in which the system executed with minimal user intervention; (2) half-autonomous mode without LTT, in which participants could provide guidance but the self-improving component was disabled; and (3) mixed-initiative mode with LTT enabled, in which the system could retrieve prior knowledge, reflect on uncertainty, and proactively solicit user feedback. Because these modes vary in both human involvement and the availability of LTT, our study does not isolate the independent effect of LTT. Since LTT depends on user feedback to accumulate and retrieve knowledge, a controlled ablation would also require standardizing user interactions, which we leave to future work.

For the first three tasks (hurricane, foot, and tornado), each participant was assigned a different autonomy mode for each case, ensuring that each participant experienced all three modes exactly once. We further balanced assignments across participants so that each mode was explored the same number of times. For the fourth task (combustion), participants were allowed to choose their preferred autonomy setting, enabling us to observe which level of control they naturally favored after gaining experience with the system. We did not include the half-cylinder case in the user study because interpreting the Q-criterion and λ_2-based vortex identification requires substantial domain knowledge in flow analysis, which could be difficult for novice participants to understand reliably.

Tasks and procedure. The study followed a four-phase workflow: introduction, exploration, task execution, and post-task survey. In the introduction phase, participants watched a short demonstration of HiLSVA using the entropy field from the Richtmyer–Meshkov instability simulation. They were introduced to the main interface and its interaction mechanisms. In the exploration phase, they were given time to freely interact with the system on the same dataset, ask questions, and become comfortable with the workflow before beginning the recorded task session.

In the task execution phase, participants completed four visualization tasks spanning both basic operations and more complex workflow

construction. The hurricane case involves slice-based scalar field visualization, histogram inspection, and contour refinement. The focus of the foot case is on isosurface rendering, external anatomy reference retrieval, and visualization refinement for clearer bone identification. The tornado case examines vector field visualization through iterative streamline exploration, trial selection, rollback, and glyph-based refinement. The combustion case involves time-varying multivariate analysis, including timestep identification, volume rendering, and animation of the `Y_OH` field, as well as isosurface animation of the `mixfrac` field. In all tasks, participants also answered several interpretation questions about the resulting visualizations, allowing us to assess both task completion and understanding of the data.

After completing the interactive tasks, participants filled out a post-study questionnaire covering background, usability, interpretability, perceived transparency, and the quality of human–agent collaboration. The questionnaire was designed as a task-specific instrument to evaluate key design goals of HiLSVA. It captures participants' subjective perceptions of system transparency, rather than providing a comprehensive evaluation of trust calibration. These survey responses, along with the recorded interaction logs and videos, were used to support our qualitative analysis of how users with varying levels of expertise engaged with HiLSVA across different autonomy settings. The detailed user study protocol is provided in the supplementary material.

Table 2: User study results. Time is reported in minutes. The first three tasks use balanced assignments across autonomy modes (FA: full-auto, HA: half-auto, MI: mixed-initiative). In Task 4 (combustion), participants selected their preferred mode, and we report the chosen mode and its execution time. The last row reports mean±std.

Pcp.	Expertise	Exp. Time	Comp. Time	FA Time	HA Time	MI Time	Task 4 Time	Task 4 Mode	Q&A Accuracy
P1	expert	15	67	13	22	24	8	MI	12 / 12
P2	scientist	9	47	6	9	15	17	MI	12 / 12
P3	novice	7	55	18	12	13	12	HA	11 / 12
P4	novice	10	38	7	7	14	10	HA	12 / 12
P5	novice	5	41	10	13	10	8	FA	12 / 12
P6	expert	4	37	9	11	9	8	FA	12 / 12
P7	novice	14	37	10	10	9	8	MI	11 / 12
P8	expert	7	51	11	16	14	10	FA	12 / 12
P9	scientist	5	27	7	6	8	6	FA	12 / 12
P10	scientist	10	54	10	15	13	16	FA	12 / 12
P11	scientist	4	51	10	11	18	12	FA	12 / 12
P12	novice	5	36	7	8	15	6	FA	11 / 12
	—	7.92±3.75	45.08±11.03	9.83±3.27	11.67±4.44	13.50±4.46	10.08±3.58	—	11.75±0.45 / 12

Task performance. As summarized in Table 2, HiLSVA supported strong task performance across all 12 participants: all participants completed the study, and accuracy on the domain-specific interpretation questions averaged 11.75 out of 12. For the first three tasks, we ensured that each task was explored the same number of times under each autonomy mode, enabling a fair comparison of execution time. Under this controlled setup, execution time was lowest in the full-auto mode (9.83 ± 3.27 min), followed by half-auto mode (11.67 ± 4.44 min), and highest in mixed-initiative mode (13.50 ± 4.46 min). Because each participant experienced all three modes, we tested these differences with a Friedman test, which was not significant ($\chi^2(2) = 5.09$, $p = 0.078$, Kendall's $W = 0.19$ [35]); Holm-corrected pairwise Wilcoxon tests likewise showed no significant pair, though full-auto versus mix-initiative had a large effect size (Cliff's $\delta = -0.51$ [14]). We therefore interpret the monotonic increase in mean execution time not as a reliable speed difference, but as a tradeoff between oversight and efficiency, where greater user involvement incurs additional interaction time while enabling verification and intervention.

Task performance also did not differ significantly across expertise levels. Grouping participants into SciVis experts ($n = 3$), domain experts ($n = 4$), and non-experts ($n = 5$), Kruskal-Wallis tests showed no significant group differences in task completion time ($H = 0.92$, $p = 0.63$) or exploration time ($H = 0.28$, $p = 0.87$), and accuracy on embedded domain-interpretation questions was uniformly high (100%, 100%, and 95% for the three groups). HiLSVA thus enables non-experts to achieve expert-comparable performance, and we observed no data hallucinations (fabricated values or non-existent fields) in the logged sessions.

Participants' autonomy choices in the final task further show that no single interaction style is universally optimal. After experiencing all three autonomy settings in the earlier tasks, participants were allowed to choose their preferred mode for the final combustion task: seven selected full-autonomous mode, two selected half-autonomous mode, and three selected mixed-initiative mode. Many participants preferred greater automation once they became familiar with the system, likely valuing efficiency for a task they felt the agent could handle reliably. At the same time, a notable subset of participants still preferred modes that preserved greater human control.

Additional results. Results on the post-study questionnaire and participant feedback are provided in Appendix A.

5 Limitations and Future Work

While HiLSVA presents a novel way to human-in-the-loop SciVis, several limitations remain. First, reliance on external LLM APIs introduces latency, despite partial mitigation through parallel execution in different virtual environments. Second, although LTT and a dynamic knowledge base enable adaptation, performance remains dependent on the quality of user feedback. In specialized scientific domains, relevant knowledge is often tacit or difficult to express in natural language, limiting effective learning. Incorporating curated domain-specific knowledge could further support complex workflows such as feature extraction. We also note that HiLSVA's adaptation is retrieval-based, which accumulates and reuses structured knowledge rather than improving the model, and our study does not isolate its gains from those attributable to increased human involvement. Stronger self-improvement could come from self-evolving agents that discover new designs [29], edit their own code [55], or harness-level optimization [38], which we leave to future work. Finally, our user study ($N = 12$) is powered to detect large within-subject effects but not fine-grained between-group or interaction effects; expertise-stratified and interaction results are therefore descriptive and indicative. Larger-scale, longitudinal studies with domain scientists are needed to confirm these patterns.

Future work will extend HiLSVA along three directions. First, while the current implementation focuses on ParaView because it provides a rich set of SciVis capabilities and supports MCP tool calls, generated scripts, and direct GUI manipulation within a shared visualization session, the overall architecture is backend-agnostic. We plan to integrate additional visualization and analysis tools to improve generality, including domains such as molecular dynamics, astrophysics, fluid simulation, and bioimage processing. Second, we aim to enhance learning through better uncertainty modeling, multimodal grounding, and long-term personalization. Prior work highlights the importance of uncertainty-aware interaction in human–agent communication [7, 8], as well as the role of multimodal human feedback in disambiguating user intent [40, 78]. Third, we will expand support for non-expert users through recommendation-based analysis assistance, collaborative multi-user environments, and online deployment. By leveraging prior workflows, domain knowledge, and retrieved examples, HiLSVA could proactively suggest candidate analysis strategies with explanations and expected outcomes, allowing users to select and refine appropriate workflows.

6 Conclusions

We have presented HiLSVA, a human-in-the-loop agentic system that repositions SciVis as a collaborative process between humans and LLM agents. Through a mixed-initiative agentic architecture, stepwise provenance tracking, and LTT adaptation, HiLSVA enables transparent, controllable, and interactive workflows that support both automation and human oversight. Our evaluation demonstrates that users at all levels of expertise, including novices, can complete non-trivial SciVis tasks with the system, while mixed-initiative interaction provides a flexible balance between efficiency and control. These findings highlight the importance of designing agentic systems that augment human reasoning rather than replace it. We believe that HiLSVA offers a practical foundation for future human-centered SciVis systems and opens new directions for integrating transparent, collaborative AI into complex scientific workflows.

Acknowledgments

This research was supported in part by the U.S. National Science Foundation through grants IIS-2101696, OAC-2104158, IIS-2401144,

and IIS-2550610, and the U.S. Department of Energy through grant DE-SC0023145. The authors thank the anonymous reviewers for their insightful comments.

REFERENCES

[1] J. P. Ahrens, B. Geveci, and C. C. Law. ParaView: An end-user tool for large-data visualization. In C. D. Hansen and C. R. Johnson, eds., *The Visualization Handbook*, chap. 36, pp. 717–731. Academic Press, 2004. doi: 10.1016/B978-012387582-2/50038-1 4

[2] K. Ai, H. Miao, Z. Li, C. Wang, and S. Liu. An evaluation-centric paradigm for scientific visualization agents. In *Proc. IEEE VIS & GenAI*, 2025. doi: 10.48550/arXiv.2509.15160 1, 3

[3] K. Ai, H. Miao, K. Tang, N. Gorski, J. Sun, G. Liu et al. SciVisAgentBench: A benchmark for evaluating scientific data analysis and visualization agents. *arXiv preprint arXiv:2603.29139*, 2026. doi: 10.48550/arXiv.2603.29139 3

[4] K. Ai, H. Miao, K. Tang, S. Liu, and C. Wang. SciVisAgentSkills: Design and evaluation of agent skills for scientific data analysis and visualization. *arXiv preprint arXiv:2606.05525*, 2026. doi: 10.48550/arXiv.2606.05525 2

[5] K. Ai, K. Tang, and C. Wang. NLI4VolVis: Natural language interaction for volume visualization via multi-LLM agents and editable 3D Gaussian splatting. *IEEE Trans. Vis. Comput. Graph.*, 32(1):46–56, 2026. doi: 10.1109/TVCG.2025.3633888 1, 2, 3

[6] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use. 1, 2

[7] G. Bansal, B. Nushi, E. Kamar, D. S. Weld, W. S. Lasecki, and E. Horvitz. Updates in human-AI teams: Understanding and addressing the performance/compatibility tradeoff. In *Proc. AAAI*, pp. 2429–2437, 2019. doi: 10.1609/aaai.v33i01.33012429 2, 9

[8] G. Bansal, J. W. Vaughan, S. Amershi, E. Horvitz, A. Fourney, H. Mozannar et al. Challenges in human-agent communication. *arXiv preprint arXiv:2412.10380*, 2024. doi: 10.48550/arXiv.2412.10380 2, 9

[9] G. Bansal, T. Wu, J. Zhou, R. Fok, B. Nushi, E. Kamar et al. Does the whole exceed its parts? the effect of AI explanations on complementary team performance. In *Proc. ACM CHI*, pp. 81:1–81:16, 2021. doi: 10.1145/3411764.3445717 2

[10] L. Bavoil, S. Callahan, P. Crossno, J. Freire, C. Scheidegger, C. Silva et al. VisTrails: enabling interactive multiple-view visualizations. In *Proc. IEEE VIS*, pp. 135–142, 2005. doi: 10.1109/VISUAL.2005.1532788 4

[11] A. Biswas, T. L. Turton, N. R. Ranasinghe, S. Jones, B. Love, W. Jones et al. VizGenie: Toward self-refining, domain-aware workflows for next-generation scientific visualization. *IEEE Trans. Vis. Comput. Graph.*, 32(1):1021–1031, 2026. doi: 10.1109/TVCG.2025.3634655 1, 2, 3

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal et al. Language models are few-shot learners. In *Proc. NeurIPS*, pp. 1877–1901, 2020. 5

[13] Z. Chen, J. Chen, S. Ö. Arik, M. Sra, T. Pfister, and J. Yoon. CoDA: Agentic systems for collaborative data visualization. *arXiv preprint arXiv:2510.03194*, 2025. doi: 10.48550/arXiv.2510.03194 2

[14] N. Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993. doi: 10.1037/0033-2909.114.3.494 9

[15] V. Dhanoa, A. Wolter, G. M. León, H.-J. Schulz, and N. Elmqvist. Agentic visualization: Extracting agent-based design patterns from visualization systems. *IEEE Comput. Graph. Appl.*, 45(6):89–90, 2025. doi: 10.1109/MCG.2025.3607741 1, 2

[16] V. Dibia. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In *Proc. ACL: System Demonstrations*, pp. 113–126, 2023. doi: 10.18653/v1/2023.acl-demo.11 2

[17] P. P. Do, C. M. Ta, and C. Wang. Benchmarking multimodal large language models for scientific visualization literacy. In *Proc. IEEE VIS (Short Papers)*, 2026. Accepted. 3

[18] P. P. Do, K. Tang, K. Ai, and C. Wang. SVLAT: Scientific visualization literacy assessment test. *arXiv preprint arXiv:2603.19000*, 2026. doi: 10.48550/arXiv.2603.19000 3

[19] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li et al. A survey on in-context learning. In *Proc. EMNLP*, pp. 1107–1128, 2024. doi: 10.18653/v1/2024.emnlp-main.64 5

[20] EgT. GMX-VMD-MCP: MCP service for GROMACS and VMD molecular dynamics simulations and visualization. https://github.com/egtai/gmx-vmd-mcp, 2025. 4

[21] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang et al. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025. doi: 10.48550/arXiv.2508.07407 2, 5

[22] K. Feng, K. Pu, M. Latzke, T. August, P. Siangliulue, J. Bragg et al. Cocoa: Co-planning and co-execution with AI agents. *arXiv preprint arXiv:2412.10999*, 2024. doi: 10.48550/arXiv.2412.10999 2, 3, 4

[23] G. Fragiadakis, C. Diou, G. Kousiouris, and M. Nikolaidou. Evaluating human-AI collaboration: A review and methodological framework. *arXiv preprint arXiv:2407.19098*, 2024. doi: 10.48550/arXiv.2407.19098 3

[24] J. Freire, D. Koop, E. Santos, and C. T. Silva. Provenance for computational tasks: A survey. *IEEE Comput. Sci. Eng.*, 10(3):11–21, 2008. doi: 10.1109/MCSE.2008.79 4

[25] N. Gorski, S. Liu, and B. Wang. TopoPilot: Reliable conversational workflow automation for topological data analysis and visualization. *arXiv preprint arXiv:2603.25063*, 2026. doi: 10.48550/arXiv.2603.25063 2, 4

[26] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla et al. Large language model based multi-agents: A survey of progress and challenges. In *Proc. IJCAI*, pp. 8048–8057, 2024. doi: 10.24963/ijcai.2024/890 2

[27] Y. He, R. Li, A. Chen, Y. Liu, Y. Chen, Y. Sui et al. Enabling self-improving agents to learn at test time with human-in-the-loop guidance. In *Proc. EMNLP: Industry Track*, pp. 1625–1653, 2025. doi: 10.18653/v1/2025.emnlp-industry.115 5

[28] E. Horvitz. Principles of mixed-initiative user interfaces. In *Proc. ACM CHI*, pp. 159–166, 1999. doi: 10.1145/302979.303030 1, 2

[29] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. In *Proc. ICLR*, vol. 2025, pp. 21344–21377, 2025. doi: 10.48550/arXiv.2408.08435 9

[30] X. Hu, T. Xiong, B. Yi, Z. Wei, R. Xiao, Y. Chen et al. OS Agents: A survey on MLLM-based agents for computer, phone and browser use. In *Proc. ACL*, pp. 7436–7465, 2025. doi: 10.18653/v1/2025.acl-long.369 1

[31] J. Hübotter, S. Bongni, I. Hakimi, and A. Krause. Efficiently learning at test-time: Active fine-tuning of LLMs. *arXiv preprint arXiv:2410.08020*, 2024. doi: 10.48550/arXiv.2410.08020 5

[32] W. Humphrey, A. Dalke, and K. Schulten. VMD: Visual molecular dynamics. *J. Mol. Graph.*, 14:33–38, 1996. doi: 10.1016/0263-7855(96)00018-5 4

[33] F. Huq, Z. Z. Wang, F. F. Xu, T. Ou, S. Zhou, J. P. Bigham et al. CowPilot: A framework for autonomous and human-agent collaborative web navigation. In *Proc. ACL: System Demonstrations*, pp. 163–172, 2025. doi: 10.18653/v1/2025.naacl-demo.17 2, 3, 4

[34] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen. From LLMs to LLM-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479*, 2024. doi: 10.48550/arXiv.2408.02479 1

[35] M. G. Kendall and B. Babington Smith. The problem of *m* rankings. *The Annals of Mathematical Statistics*, 10(3):275–287, 1939. doi: 10.1214/aoms/1177732186 9

[36] J. Y. Koh, S. McAleer, D. Fried, and R. Salakhutdinov. Tree search for language model agents. *arXiv preprint arXiv:2407.01476*, 2024. doi: 10.48550/arXiv.2407.01476 2

[37] H. Lai, X. Liu, I. L. Iong, S. Yao, Y. Chen, P. Shen et al. AutoWebGLM: A large language model-based web navigating agent. In *Proc. ACM KDD*, pp. 5295–5306, 2024. doi: 10.1145/3637528.3671620 1

[38] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. doi: 10.48550/arXiv.2603.28052 9

[39] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS*, pp. 9459–9474, 2020. 5

[40] T. Li, G. Li, J. Zheng, P. Wang, and Y. Li. MUG: Interactive multimodal grounding on user interfaces. In *Proc. ACL Findings*, pp. 231–251, 2024. doi: 10.18653/v1/2024.findings-eacl.17 9

[41] J. Liu, J. Hao, C. Zhang, and Z. Hu. WEPO: Web element preference optimization for LLM-based web navigation. In *Proc. AAAI*, pp. 26614–26622, 2025. doi: 10.1609/aaai.v39i25.34863 1

[42] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang et al. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977*, 2024. doi: 10.48550/arXiv.2409.02977 1

[43] S. Liu, H. Miao, and P.-T. Bremer. ParaView-MCP: An autonomous visualization agent with direct tool use. In *Proc. IEEE VIS (Short Papers)*, pp. 61–65, 2025. doi: 10.48550/arXiv.2505.07064 1, 2, 4

[44] S. Liu, H. Miao, Z. Li, M. Olson, V. Pascucci, and P.-T. Bremer. AVA: towards autonomous visualization agents through visual perception-driven

decision-making. *Comput. Graph. Forum*, 43(3):e15093, 2024. doi: 10.1111/cgf.15093 1, 2

[45] T. Masterman, S. Besen, M. Sawtell, and A. Chao. The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024. doi: 10.48550/arXiv.2404.11584 2

[46] M. Mathai, M. Han, J. Knowles, V. A. Mateevitsi, S. Rizzi, and H. Childs. NL2SciVis: A benchmark for natural language to scientific visualization. In *Proc. EuroVis (Short Papers)*, 2026. doi: 10.2312/evs.20261017 3

[47] H. Miao, Z. Li, K. Ai, K. Tang, C. Wang, P.-T. Bremer et al. Toward AI VIS co-scientists: A general and end-to-end agent harness for solving complex data visualization tasks. *arXiv preprint arXiv:2605.21825*, 2026. doi: 10.48550/arXiv.2605.21825 2, 3

[48] H. Miao and S. Liu. BioImage-Agent. `https://github.com/LLNL/bioimage-agent`. 2, 4

[49] H. Mozannar, G. Bansal, C. Tan, A. Fourney, V. Dibia, J. Chen et al. Magentic-UI: Towards human-in-the-loop agentic systems. *arXiv preprint arXiv:2507.22358*, 2025. doi: 10.48550/arXiv.2507.22358 1, 2, 3, 4

[50] A. Narechania, S. Guo, E. Koh, A. Endert, and J. Hoffswell. Utilizing provenance as an attribute for visual data analysis: A design probe with provenancelens. *IEEE Trans. Vis. Comput. Graph.*, 31(10):8452–8465, 2025. doi: 10.1109/TVCG.2025.3571708 4

[51] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr. Autonomous evaluation and refinement of digital agents. *arXiv preprint arXiv:2404.06474*, 2024. doi: 10.48550/arXiv.2404.06474 2

[52] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023. doi: 10.48550/arXiv.2302.06590 2

[53] T. Peterka, T. Mallick, O. Yildiz, D. Lenz, C. Quammen, and B. Geveci. ChatVis: Large language model agent for generating scientific visualizations. In *Proc. IEEE LDAV*, pp. 22–32, 2025. doi: 10.1109/LDAV68558.2025.00007 1, 2, 4

[54] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proc. ICLR*, 2023. doi: 10.48550/arXiv.2307.16789 2

[55] M. Robeyns, M. Szummer, and L. Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025. doi: 10.48550/arXiv.2504.15228 9

[56] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro et al. Toolformer: Language models can teach themselves to use tools. In *Proc. NeurIPS*, pp. 68539–68551, 2023. doi: 10.5555/3666122.3669119 2

[57] J. Senoner, S. Schallmoser, B. Kratzwald, S. Feuerriegel, and T. Netland. Explainable AI improves task performance in human–AI collaboration. *Sci. Rep.*, 14(1):31150, 2024. doi: 10.1038/s41598-024-82501-9 1, 2

[58] Y. Shao, V. Samuel, Y. Jiang, J. Yang, and D. Yang. Collaborative Gym: A framework for enabling and evaluating human-agent collaboration. *arXiv preprint arXiv:2412.15701*, 2024. doi: 10.48550/arXiv.2412.15701 3

[59] B. Shneiderman and P. Maes. Direct manipulation vs. interface agents. *Interactions*, 4(6):42–61, 1997. doi: 10.1145/267505.267514 1, 2

[60] N. Sofroniew, T. Lambert, G. Bokota, J. Nunez-Iglesias, P. Sobolewski, A. Sweet et al. napari: A multi-dimensional image viewer for Python, 2025. doi: 10.5281/zenodo.3555620 4

[61] P. Stone and M. Veloso. Multiagent systems: A survey from a machine learning perspective. *Auton. Robots*, 8(3):345–383, 2000. doi: 10.1023/A:1008942012299 2

[62] J. Sun, D. Lenz, T. Peterka, and H. Yu. SASAV: Self-directed agent for scientific analysis and visualization. *arXiv preprint arXiv:2604.03406*, 2026. doi: 10.48550/arXiv.2604.03406 2, 3

[63] Z. Sun, Z. Liu, Y. Zang, Y. Cao, X. Dong, T. Wu et al. SEAgent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025. doi: 10.48550/arXiv.2508.04700 2, 5

[64] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang et al. Human-in-the-loop software development agents. In *Proc. IEEE/ACM ICSE: SEIP*, pp. 342–352, 2025. doi: 10.1109/MSR66628.2025.00112 1, 3

[65] Y. Talebirad and A. Nadiri. Multi-Agent Collaboration: Harnessing the power of intelligent LLM agents. *arXiv preprint arXiv:2306.03314*, 2023. doi: 10.48550/arXiv.2306.03314 2

[66] J. Z. Tam, P. Grosset, D. Banesh, N. Ramachandra, T. L. Turton, and J. Ahrens. InferA: A smart assistant for cosmological ensemble data. In *Proc. ACM/IEEE SC Workshops*, pp. 20–28, 2025. doi: 10.1145/3731599.3767342 2, 3

[67] M. Tambe. Implementing agent teams in dynamic multiagent environments. *Appl. Artif. Intell.*, 12(2-3):189–210, 1998. doi: 10.1080/088395198117820 2

[68] K. Tang, K. Ai, J. Han, and C. Wang. TexGS-VolVis: Expressive scene editing for volume visualization via textured Gaussian splatting. *IEEE Trans. Vis. Comput. Graph.*, 32(1):933–943, 2026. doi: 10.1109/TVCG.2025.3634643 3

[69] K. Tang, S. Yao, and C. Wang. iVR-GS: Inverse volume rendering for explorable visualization via editable 3D Gaussian splatting. *IEEE Trans. Vis. Comput. Graph.*, 31(6):3783–3795, 2025. doi: 10.1109/TVCG.2025.3567121 1

[70] J. Tierny, G. Favelier, J. A. Levine, C. Gueunet, and M. Michaux. The topology toolkit. *IEEE Trans. Vis. Comput. Graph.*, 24(1):832–842, 2017. doi: 10.1109/TVCG.2017.2743938 4

[71] J. E. Vonderhorst, K. Ai, H. Miao, S. Liu, and C. Wang. Exploring LLM agent designs and interaction modalities for scientific visualization. *arXiv preprint arXiv:2604.27996*, 2026. doi: 10.48550/arXiv.2604.27996 2

[72] C. Wang and J. Han. DL4SciVis: A state-of-the-art survey on deep learning for scientific visualization. *IEEE Trans. Vis. Comput. Graph.*, 29(8):3714–3733, 2023. doi: 10.1109/TVCG.2022.3167896 1

[73] Y. Wang, B. Pan, K. Wang, H. Liu, J. Mao, Y. Liu et al. IntuiTF: MLLM-guided transfer function optimization for direct volume rendering. *arXiv preprint arXiv:2506.18407*, 2025. doi: 10.48550/arXiv.2506.18407 2

[74] K. Werder, B. Ramesh, and R. S. Zhang. Establishing data provenance for responsible artificial intelligence systems. *ACM Trans. Manag. Inf. Syst.*, 13(2):22:1–22:33, 2022. doi: 10.1145/3503488 4

[75] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. *arXiv preprint arXiv:2308.08155*, 2023. doi: 10.48550/arXiv.2308.08155 2

[76] Z. Wu, C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao et al. OS-Copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024. doi: 10.48550/arXiv.2402.07456 1, 2

[77] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong et al. The rise and potential of large language model based agents: A survey. *Sci. China Inf. Sci.*, 68(2):121101, 2025. doi: 10.1007/s11432-024-4222-0 2

[78] Y. Yang, A. G. Zhang, S. Oney, and A. Y. Wang. SPARK: Real-time monitoring of multi-faceted programming exercises. In *Proc. IEEE VL/HCC*, pp. 81–92, 2025. doi: 10.1109/VL-HCC65237.2025.00018 9

[79] Z. Yang, Z. Zhou, S. Wang, X. Cong, X. Han, Y. Yan et al. MatPlotAgent: Method and evaluation for LLM-based agentic scientific data visualization. In *Proc. ACL Findings*, pp. 11789–11804, 2024. doi: 10.18653/v1/2024.findings-acl.701 2

[80] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. doi: 10.48550/arXiv.2406.12045 3

[81] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao et al. Tree of Thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS*, pp. 11809–11822, 2023. doi: 10.5555/3666122.3666639 2

[82] C. Zhang, H. Huang, C. Ni, J. Mu, S. Qin, S. He et al. UFO2: The desktop agentOS. *arXiv preprint arXiv:2504.14603*, 2025. doi: 10.48550/arXiv.2504.14603 1, 2

[83] W. Zhang and J. Tao. Automatic semantic alignment of flow pattern representations for exploration with large language models. *IEEE Trans. Vis. Comput. Graph.*, 32(1):835–845, 2026. doi: 10.1109/TVCG.2025.3634650 3

[84] Z. Zhang and A. Zhang. You only look at screens: Multimodal chain-of-action agents. In *Proc. ACL Findings*, pp. 3132–3149, 2024. doi: 10.18653/v1/2024.findings-acl.186 2

[85] Y. Zhao, J. Wang, L. Xiang, X. Zhang, Z. Guo, C. Turkay et al. LightVA: Lightweight visual analytics with LLM agent-based task planning and execution. *IEEE Trans. Vis. Comput. Graph.*, 31(9):6162–6177, 2024. doi: 10.1109/TVCG.2024.3496112 1

A Additional User Study Results

Quantitative results. We evaluated the accessibility and perceived effectiveness of HiLSVA through a post-study questionnaire. After completing the tasks, all participants provided ratings on a 1–5 Likert scale for 14 questions, along with qualitative feedback on the system's strengths, challenges, and potential improvements (see the supplementary material). The aggregated results are shown in Figure 1.

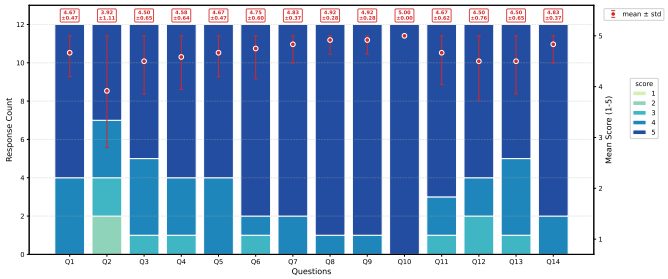

Figure 1: Survey responses to 14 post-study questions, along with mean scores (on a 1–5 scale) and standard deviations.

Overall, participants reported highly positive experiences with the system, with an average score of 4.66 across all questions. Collapsing the two expert groups ($n = 7$), experts rated the system slightly lower than non-experts (4.51 vs. 4.87; Mann-Whitney $p = 0.080$, Cliff's $\delta = -0.63$), consistent with non-experts benefiting more from agent assistance while expert users judged it more critically. Ratings of efficiency and overall effectiveness were consistently strong, with Q1 (task efficiency) receiving 4.67±0.47. The system's integration and learnability were also well received, with Q3 (system integration) at 4.50±0.65 and Q4 (ease of learning) at 4.58±0.64, indicating that participants quickly understood and adopted the workflow.

Several questions focused on human-agent interaction and control. Participants expressed a strong sense of control over the workflow, with Q6 (ability to review and approve actions) rated at 4.75±0.60 and Q7 (ability to adjust autonomy levels) at 4.83±0.37. Transparency-related aspects received some of the highest scores in the study, including Q8 (clarity of system feedback, 4.92±0.28), Q9 (usefulness of stepwise provenance for understanding system behavior, 4.92±0.28), and Q10 (ability to revisit previous steps, 5.00±0.00). These results suggest that the system's stepwise planning and provenance-tracking mechanisms effectively supported users' understanding and oversight.

The mixed-initiative design was also positively evaluated. Participants rated the combination of natural-language interaction and direct manipulation highly (Q11, 4.67±0.62) and agreed that the system could recognize uncertainty and appropriately request guidance (Q12, 4.50±0.76). In addition, Q13 (benefit from accumulated knowledge) received 4.50±0.65, indicating that the LTT mechanism was noticeable and useful during interaction. The overall interface usability was similarly strong, with Q14 rated at 4.83±0.37.

The lowest score was observed for Q2 (ability to complete tasks without assistance), with 3.92±1.11. The larger variance suggests differing expectations across participants, particularly between experts and novices. This is expected, as the four tasks involve non-trivial SciVis techniques that are challenging without prior experience. Despite this, all participants completed the tasks with high accuracy when the system provided support. This aligns with HiLSVA's design goal as a collaborative system, where human involvement remains integral while the agent lowers barriers to complex visualization workflows.

Qualitative feedback. Participants across all levels of expertise provided insightful feedback on the strengths and limitations of HiLSVA. A common theme among all groups was the value of *human–agent collaboration*. Several participants highlighted the system's ability to support iterative refinement, with a SciVis expert highlighting "*the way that user and agent can work together to refine the visualization*," and a domain scientist emphasizing the usefulness of step-by-step planning for first-time users.

Another frequently mentioned strength was the system's *interactivity and controllability*. Domain scientists emphasized that the system is "*controllable for visualization through text description*," highlighting the value of expressing visualization intent without requiring detailed knowledge of visualization software or scripting. Participants also highlighted the mixed-initiative interaction, particularly the agent's ability to request guidance when uncertain, described as "*a very cool feature*." In addition, users appreciated the integration of multiple interaction mechanisms, including both code-based execution and direct tool control through MCP. They also valued the system's autonomous capabilities, particularly its ability to integrate external tools and information sources within a unified workflow, reducing context switching and supporting iterative refinement.

Despite these strengths, participants reported challenges related to *latency and system responsiveness*. Multiple users noted that the system could be slow sometimes, with comments such as "*LLM takes too long to respond*" and "*the latency issue makes the visualization procedure pretty long*." In addition, several participants mentioned issues with system stability, including GUI inconsistencies and occasional crashes.

Feedback also revealed opportunities for improving *information presentation*. Some expert users found the system outputs overly verbose, noting that "*output always contains too much information without highlighting*," and suggested mechanisms to prioritize critical information. Novice users, on the other hand, expressed a need for more guidance, including clearer explanations of visualization concepts and domain-specific terminology. While tasks that require greater analysis and domain knowledge can benefit from stronger human oversight, novice users may still prefer higher autonomy levels because they lack the expertise to guide the workflow.

Finally, participants suggested improvements to *workflow efficiency and usability*, including better session management, voice interaction, and more expert-curated default settings. In particular, several novice users requested presets derived from expert knowledge to effectively create visualizations without requiring domain expertise.

These observations reveal several practical limitations and opportunities for improvement. Latency was primarily associated with planning and external LLM response times. GUI inconsistencies were caused by known synchronization limitations in ParaView that would require backend-level modifications to address fully. Conflicting operations typically caused occasional crashes, although the containerized architecture enables straightforward session recovery. User feedback on verbose outputs motivated a redesign that suppresses intermediate reasoning and tool-calling traces in favor of concise analytical results. Finally, the differing needs of novice and expert users suggest opportunities for adaptive guidance, recommendations, and expert-derived presets that better support complex analytical tasks.